

Lab 1:

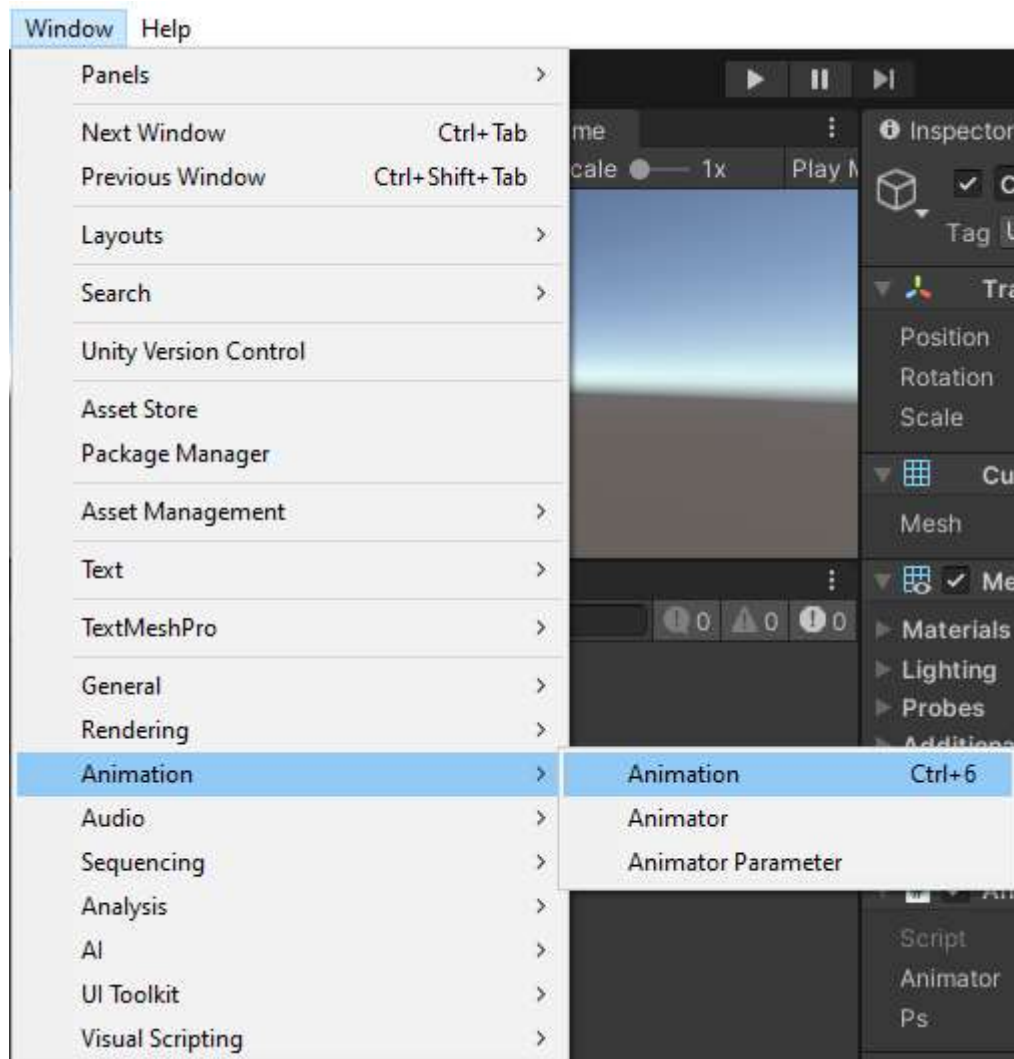
Game Design:

Create a 3D environment with particle effects, Materials, and Animation.

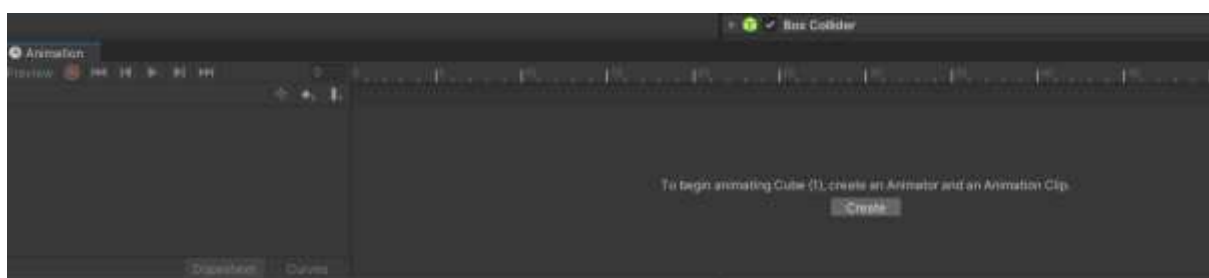
Solution :

Steps:

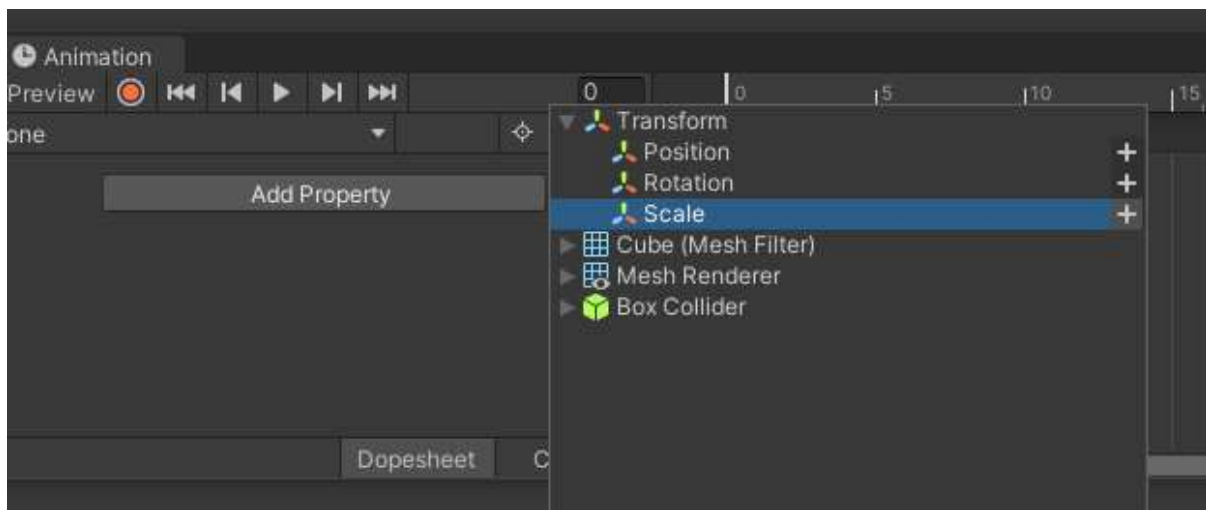
1. Create 3D environment cube & Plane
2. Select Cube -> click windows->animation->animation



2. click create ->save clip as One

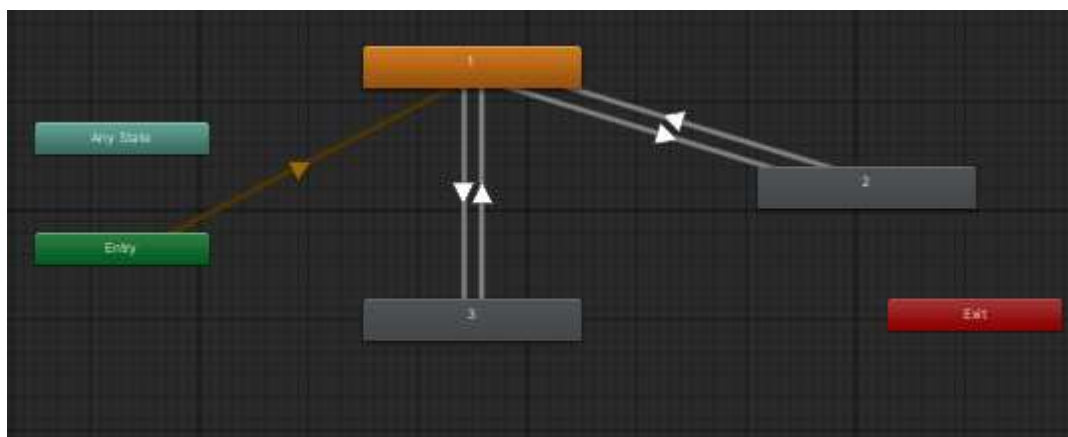


3. Click Add property, Transform, Scale click +



4. Move white line and add keys repeat same for rotation and Position

5. Open animator controller create parameters a,b and create State diagram



6. Add particle system as child of cube

7. create c# script

```
using UnityEngine;

public class AnimationController : MonoBehaviour
{
    public Animator animator;
    public ParticleSystem ps;
    void Start ()
    {
        animator=GetComponent<Animator>();
        ps=GetComponentInChildren<ParticleSystem>();
    }

    void Update()
    {
    }
```

```
// Animation 1
if (Input.GetKeyDown(KeyCode.Alpha1))
{
    animator.SetBool("a", true);
}

if (Input.GetKeyUp(KeyCode.Alpha1))
{
    animator.SetBool("a", false);
}

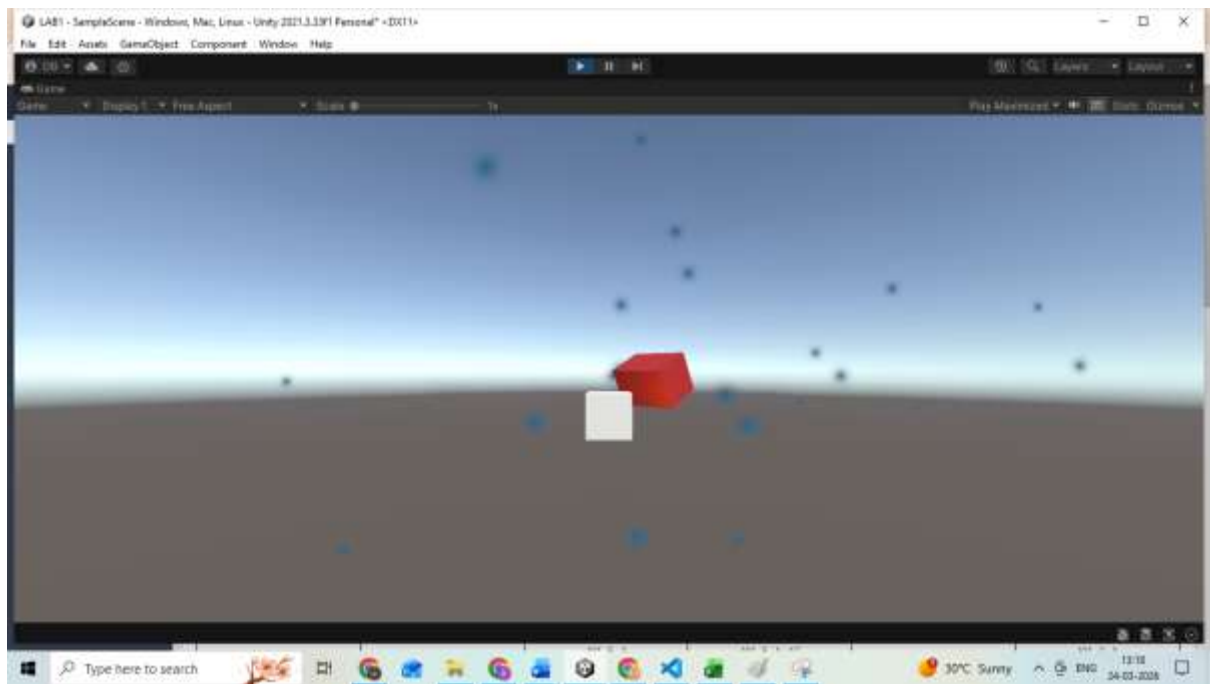
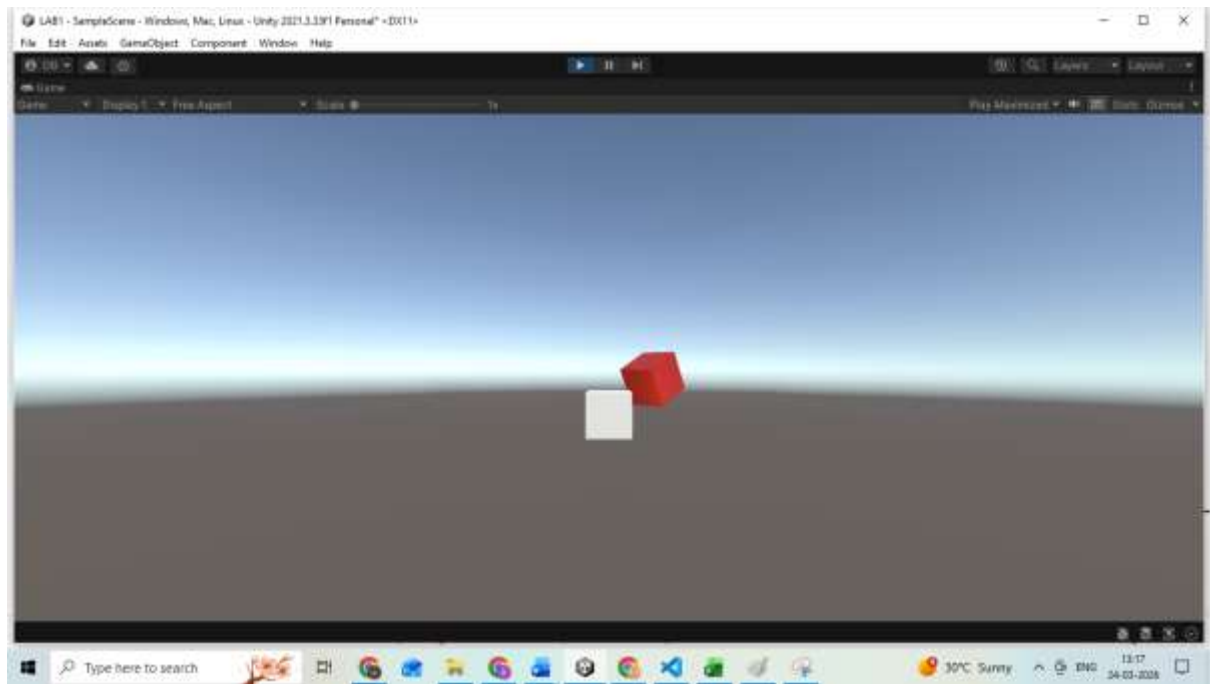
// Animation 2
if (Input.GetKeyDown(KeyCode.Alpha2))
{
    animator.SetBool("b", true);
}

if (Input.GetKeyUp(KeyCode.Alpha2))
{
    animator.SetBool("b", false);
}

//Particle system
if (Input.GetKeyDown(KeyCode.P))
{
    if (ps.isPlaying)
    {
        ps.Stop();
    }
    else
    {
        ps.Play();
    }
}

}
```

Output:



LAB 2: Basic Player Movement:

Create a player character as a 3D model and enable transformative actions such as moving left/right and forward/backward.

Solution:

1. Open Unity → New 3D Scene
2. In **Hierarchy**:
 - Right-click → **3D Object** → **Plane** (Ground)
 - Right-click → **3D Object** → **Capsule** (Player)
 - Right-click → **3D Object** → **Cube** (Obstacle)
3. Select **Cube**
4. In **Inspector** → **Tag** → **Add Tag**
5. Click + → **create "Obstacle"**
6. Go back → assign tag **Obstacle** to Cube
7. Already has **Capsule Collider**
 - Add **Rigidbody** : Click **Add Component** → **Rigidbody**
8. Right-click in **Project** → **Create** → **C# Script**

```
using System.Collections;

using System.Collections.Generic;

using UnityEngine;

public class Lab2 : MonoBehaviour

{

    public float speed = 5f;

    void Update()

    {

        // Get input from keyboard (old input system)

        float moveX = Input.GetAxis("Horizontal"); // A/D or Left/Right arrow

        float moveZ = Input.GetAxis("Vertical"); // W/S or Up/Down arrow

        // Movement vector
```

```
Vector3 move = new Vector3(moveX, 0f, moveZ);

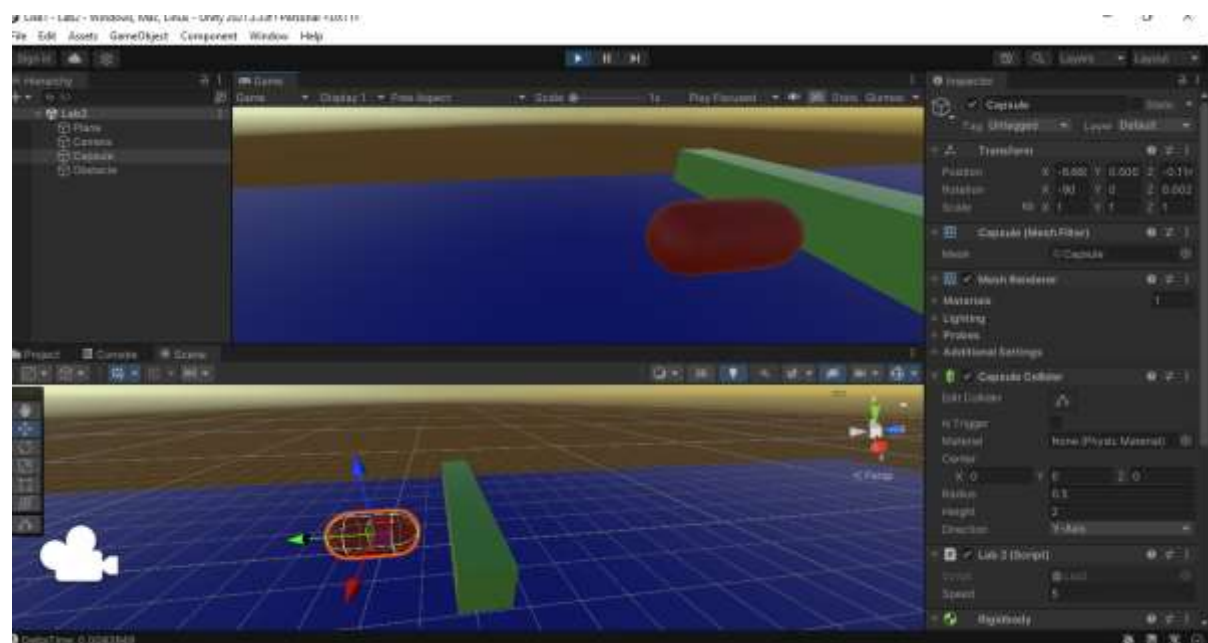
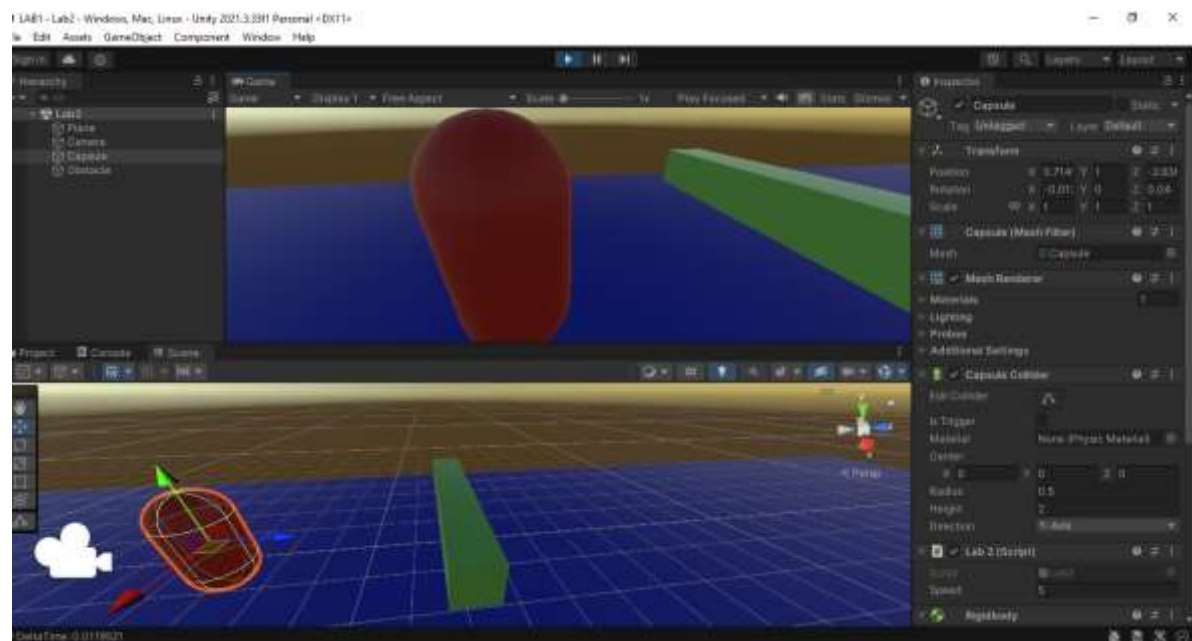
// Move the player

transform.Translate(move * speed * Time.deltaTime);
}

void OnCollisionEnter(Collision collision)
{
    // Check if collided object has tag "Obstacle"
    if (collision.gameObject.CompareTag("Obstacle"))
    {
        Debug.Log("Obstacle Found!");
    }
}
}
```

9. Attach script to capsule

Output:



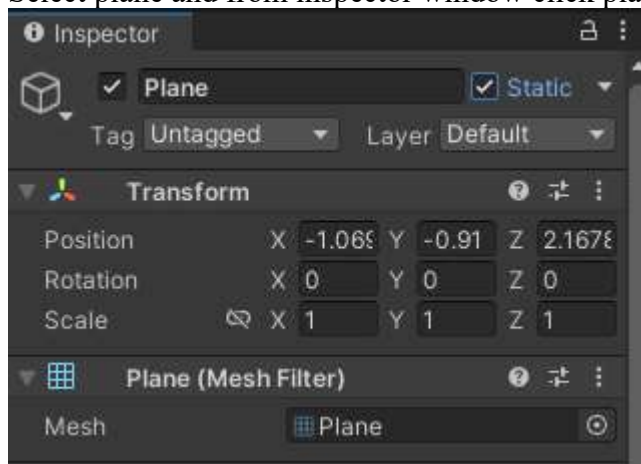
LAB 3:

Create AI in Unity

Create an AI character as a 3D model and enable AI to follow other character.

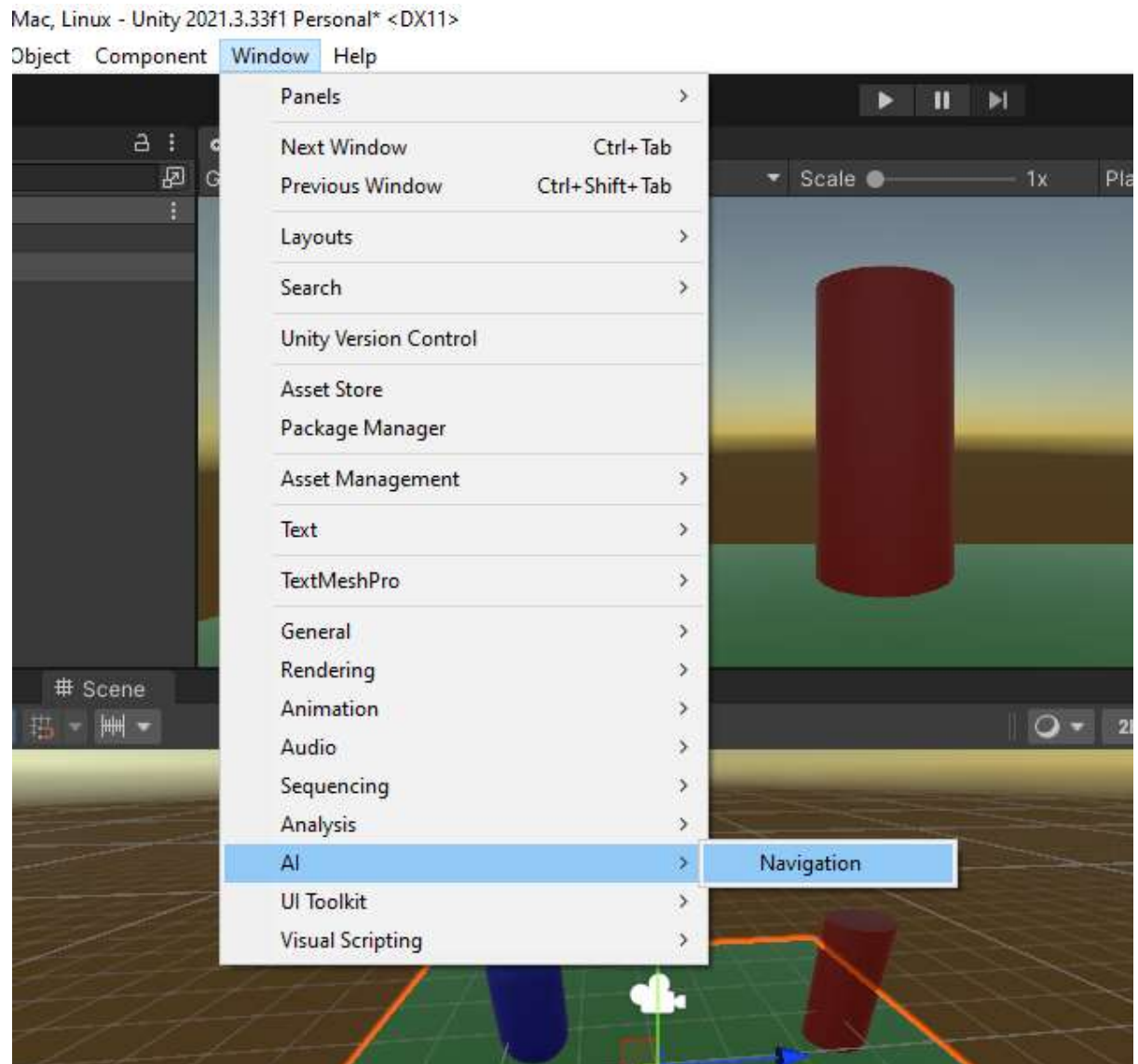
Solution:

1. Add plane 3D object
2. Add Capsule as 3D object and name it Player
3. Add other 3D object like cylinder as call it AI
4. Add Script from lab 2 to player
5. Select plane and from inspector window click plane as static



6. Add NavMesh to plane, its area for which AI is enabled , NavMesh system mainly uses the **A* Search Algorithm** for AI pathfinding.

- Click windows → AI → Navigation



- Near to inspector window navigation will open click bake u will find blue colour on plane indicates NavMesh added

7. click ai and add NavMesh agent as component and create script here use name LAB3 attach to AI



Code LAB 3.cs :

```
using UnityEngine;

using UnityEngine.AI;

public class Lab3 : MonoBehaviour
{
    public Transform player;

    public float roamRadius = 10f;

    public float detectionRange = 8f;
```

```
NavMeshAgent agent;

Vector3 roamPoint;

void Start()
{
    agent = GetComponent<NavMeshAgent>();

    SetNewRoamPoint();
}

void Update()
{
    float distance = Vector3.Distance(transform.position, player.position);

    if (distance <= detectionRange)
    {
        agent.SetDestination(player.position);
    }
    else
    {
        if (!agent.pathPending && agent.remainingDistance < 0.5f)
        {
            SetNewRoamPoint();
        }
    }
}
```

```

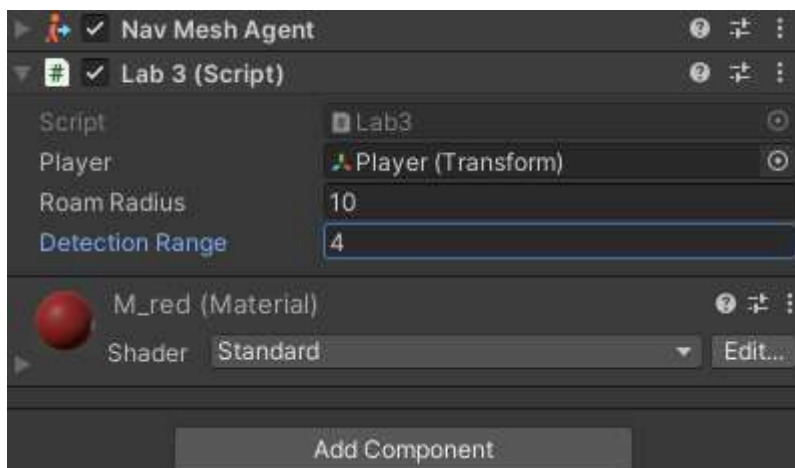
void SetNewRoamPoint()
{
    Vector3 randomDirection = Random.insideUnitSphere * roamRadius;
    randomDirection += transform.position;

    NavMeshHit hit;
    NavMesh.SamplePosition(randomDirection, out hit, roamRadius, 1);
    roamPoint = hit.position;

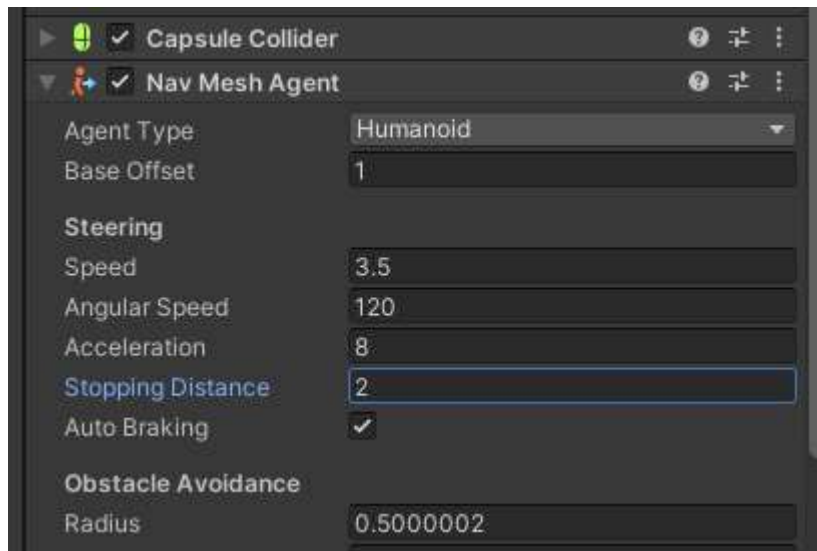
    agent.SetDestination(roamPoint);
}
}

```

Drag player to place of transform this way we get reference of player and at each frame/second, we will get current position of player you can reduce range of detection

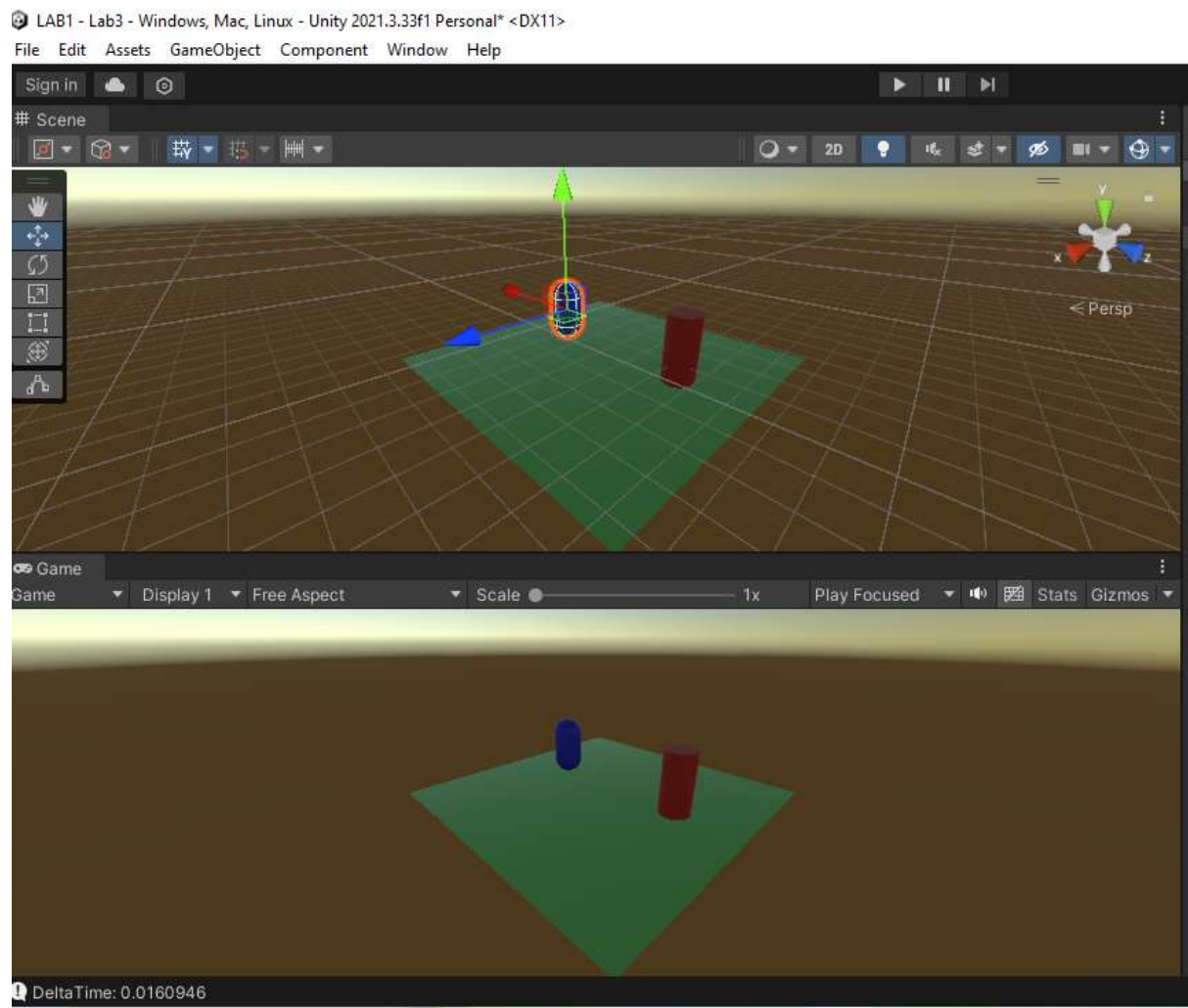


- Once AI detected player to avoid overlap in AI Navmesh Agent set stopping distance
- Note:
 - **1 Unity unit $\approx$ 1 meter**



Output:

Initial position of Player and AI



AI Detected player and followed

